2025秋计算机系统硬件基础第5次作业-张睿恒-2024302182001

T4.6&4.7

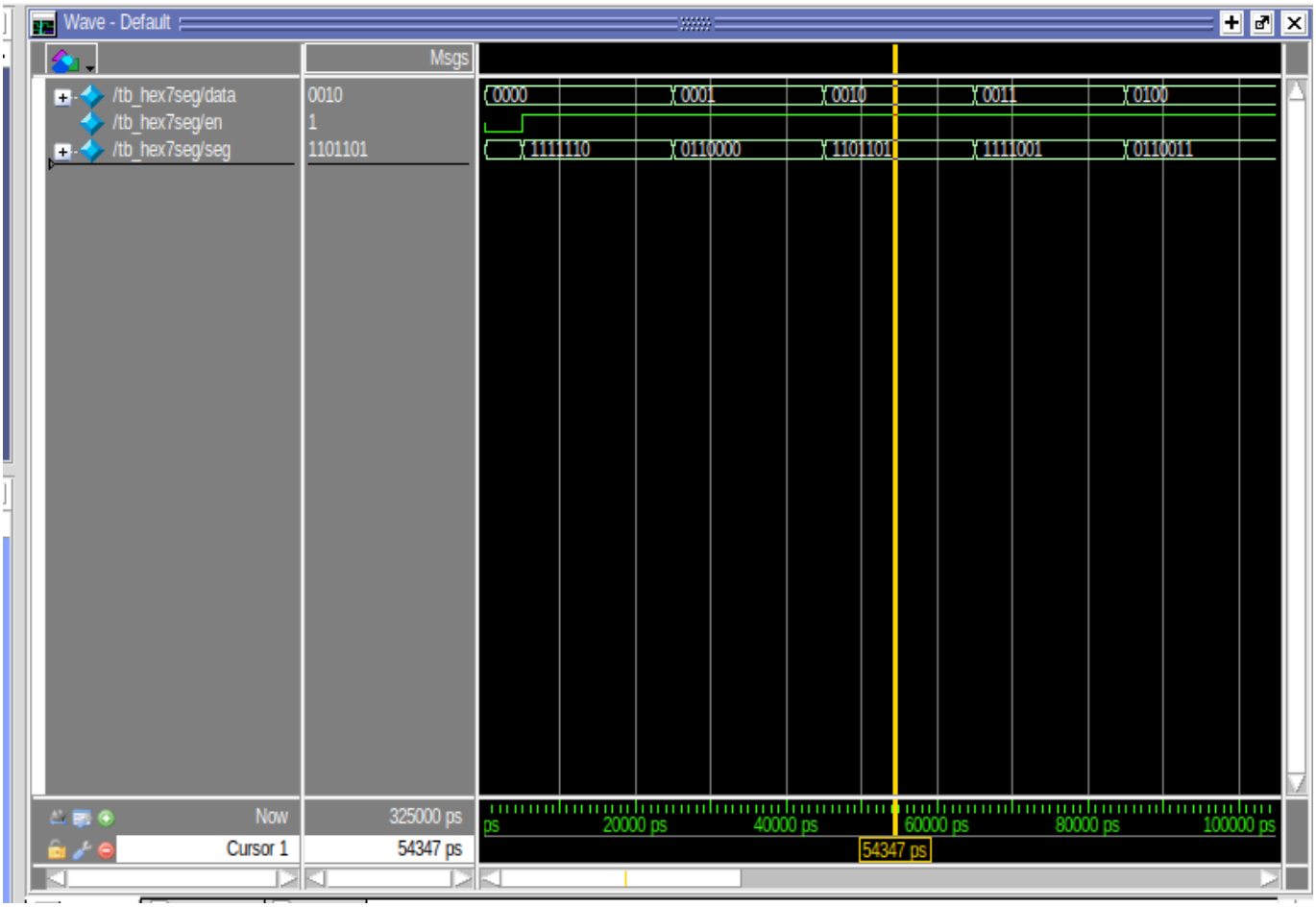
思路：采用带有always的组合电路。每当data改变都会引起输出seg的变化。

```
module hex7seg (
    input logic [3:0] data,
    input logic en,
    output logic [6:0] seg
);

always_comb begin
    if(!en) begin
        seg = 7'b0000000;
    end
    else begin
        case(data)
            4'h0: seg = 7'b1111110;
            4'h1: seg = 7'b0110000;
            4'h2: seg = 7'b1101101;
            4'h3: seg = 7'b1111001;
            4'h4: seg = 7'b0110011;
            4'h5: seg = 7'b1011011;
            4'h6: seg = 7'b0011111;
            4'h7: seg = 7'b1110000;
            4'h8: seg = 7'b1111111;
            4'h9: seg = 7'b1110011;
            4'hA: seg = 7'b1110111;
            4'hB: seg = 7'b0011111;//lower case b
            4'hC: seg = 7'b1001110;
            4'hD: seg = 7'b0111101;//lower case d
            4'hE: seg = 7'b1001111;
            4'hF: seg = 7'b1000111;
            default: seg = 7'b0000000;
        endcase
    end
end

endmodule
```

给出对应的测试程序：

```
timeunit 1ns;
time precision 1ps;

module tb_hex7seg();

logic [3:0] data;
logic en;
logic [6:0] seg;

hex7seg dut(
    .data(data),
    .en(en),
    .seg(seg)
);

initial begin
    data = 4'b0;
    en = 0;
    #5;
    en = 1;
    for(int i=0;i<16;++i) begin
        data = i;
        #20;
    end

    en = 0;
end

endmodule
```

给出仿真截图



T4.18

思路：先根据真值表写出对应的最小项表达式：

$$Y = A\dot{B}\dot{C}\dot{D} + A\dot{B}CD + AB\dot{C}\dot{D} + ABCD$$

然后照着写就行。

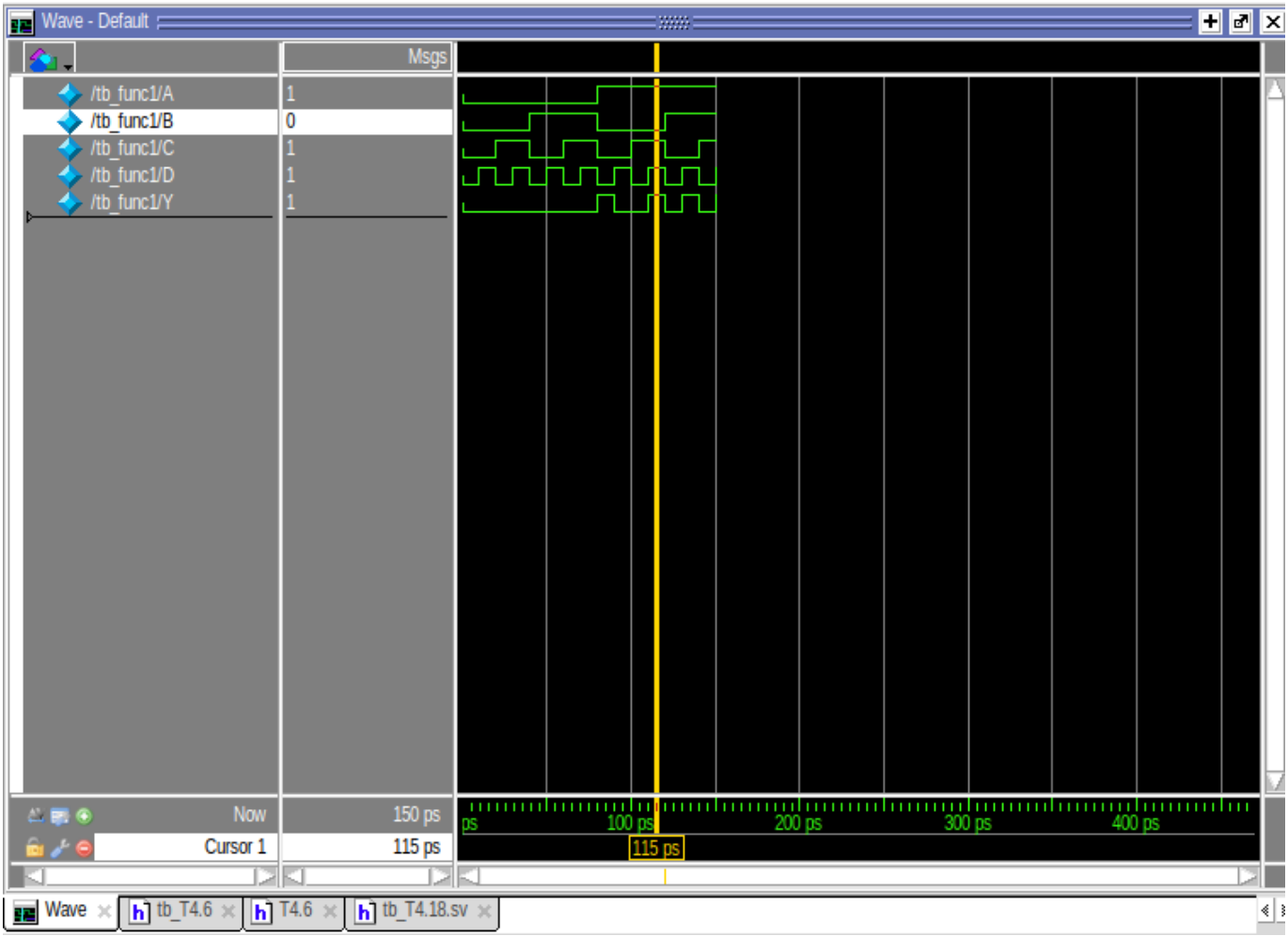
```
module func1(
    input logic A,
    input logic B,
    input logic C,
    input logic D,
    output logic Y
);


assign Y = (A&~B&~C&~D) |
           (A&~B&C&D) |
           (A&B&~C&D) |
           (A&B&~C&D) |
           (A&B&C&D);

endmodule
```

给出对应的测试程序

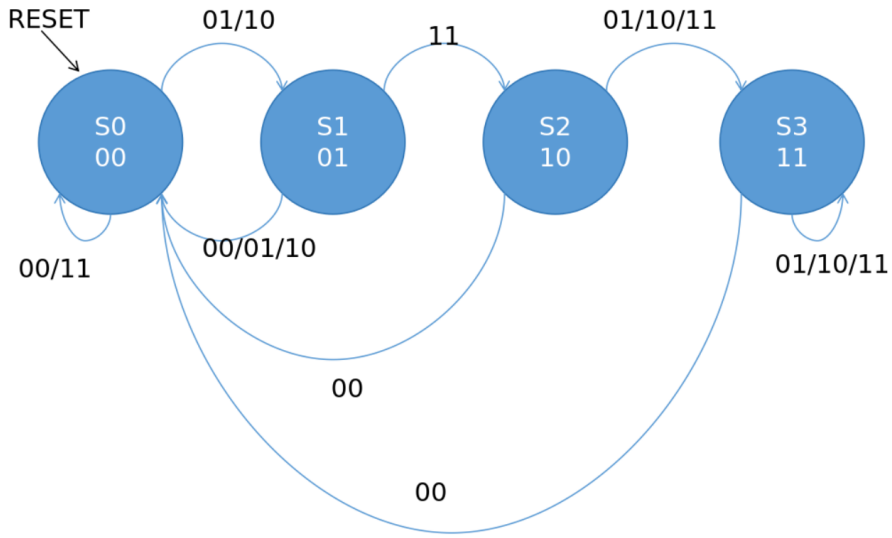
```
timeunit 10ps;
module tb_func1();

logic A,B,C,D,Y;

func1 dut(
    .A(A),
    .B(B),
    .C(C),
    .D(D),
    .Y(Y)
);

initial begin
    A=0;B=0;C=0;D=0;
    #1;
    D=1;
    #1;
    C=1;D=0;
    #1;
    D=1;
    #1;
    B=1;C=0;D=0;
    #1;
    D=1;
    #1;
    C=1;D=0;
    #1;
    D=1;
    #1;
    A=1;B=0;C=0;D=0;
    #1;
    D=1;
    #1;
    C=1;D=0;
    #1;
    C=1;D=1;
    #1;
    B=1;C=0;D=0;
    #1;
    D=1;
    #1
    C=1;D=0;
    #1;
    D=1;
end
```

```
endmodule
```

给出仿真截图



T4.24



T4.30

思路：这个题感觉不是很难，算是一个有限状态机的模板题。按照书上的描述用分别用控制灯光和转换模式的两个有限状态机写就可以了。

```
module mode(
    input logic p,
    input logic r,
    input logic clk,
    input logic reset,
    output logic m
);

typedef enum logic[0:0] {s0,s1} state1;
state1 now,next;

always_ff @(posedge clk,posedge reset) begin
    if(reset == 1) now <= s0;
    else now <= next;
end

always_comb    begin
    if(now == s0) begin
        if(p == 1) next = s1;
        else next = s0;
    end

    if(now == s1) begin
        if(r == 1) next = s0;
        else next = s1;
    end
end

assign M = (now == s1);

endmodule


module light(
    input logic ta,
    input logic tb,
    input logic m,
    input logic reset,
    input logic clk,
    output logic [1:0] la,
    output logic [1:0] lb
);

typedef enum logic [1:0] {s0,s1,s2,s3} state;
state now,next;
```

```
always_ff @(posedge clk,posedge reset) begin
    if(reset == 1) now <= s0;
    else now = next;
end

always_comb begin
    case(now)
        s0: begin
            la = 2'b10;
            lb = 2'b00;
        end
        s1: begin
            la = 2'b01;
            lb = 2'b00;
        end
        s2: begin
            la = 2'b00;
            lb = 2'b10;
        end
        s3: begin
            la = 2'b00;
            lb = 2'b01;
        end
        default: begin
            la = 2'b00;
            lb = 2'b00;
        end
    endcase
end

always_comb begin
    case(now)
        s0: if(ta == 0) next = s1;
            else next = s0;
        s1: next = s2;
        s2: if((tb == 0) &(m == 0) ) next = s3;
            else next = s2;
        s3: next = s0;
    endcase
end

endmodule


module controller(
    input logic p,
    input logic r,
    input logic reset,
```

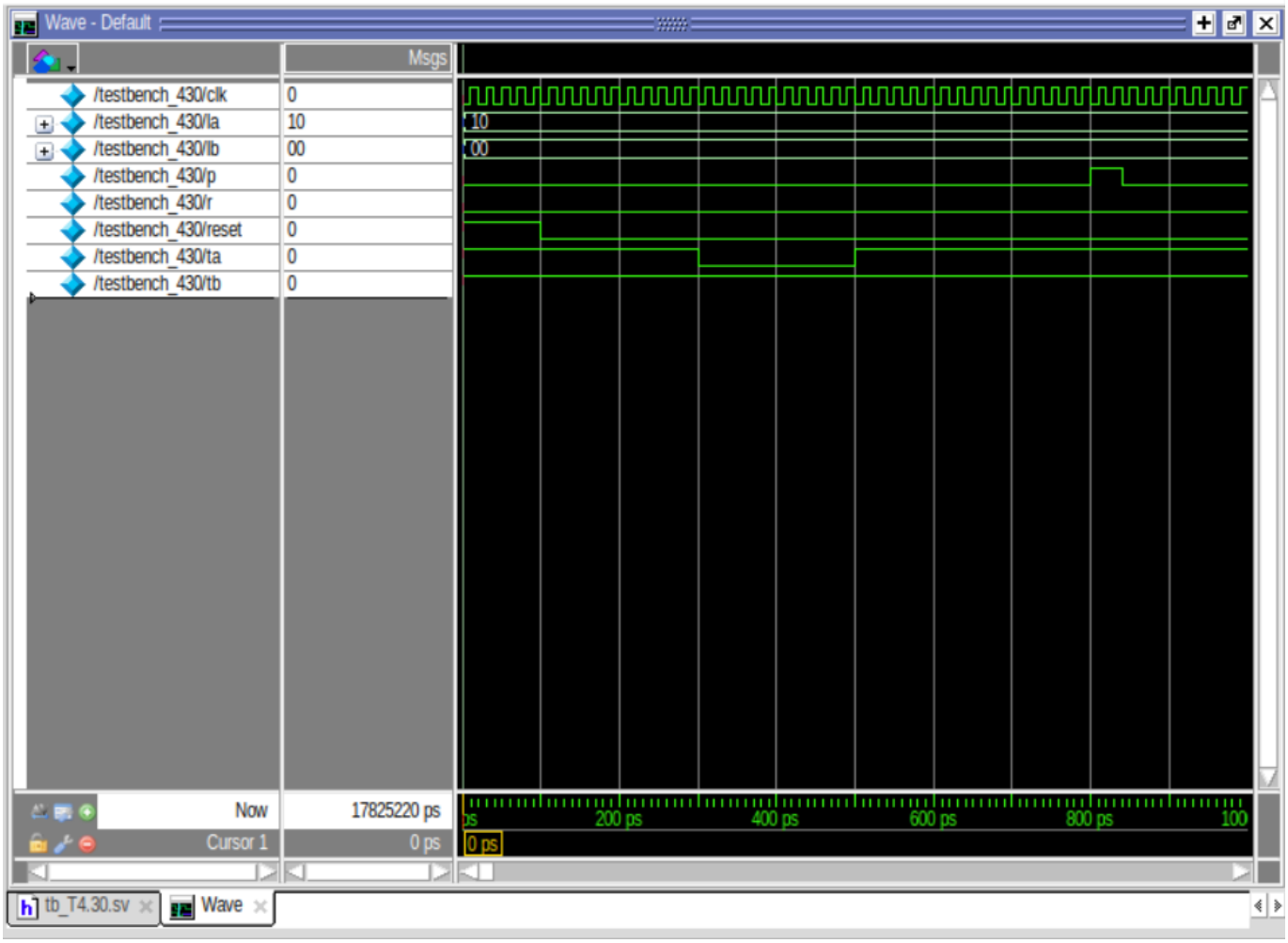
```
    input logic clk,
    input logic ta,
    input logic tb,
    output logic [1:0] la,
    output logic [1:0] lb
);

logic m;
mode mode1(
    .p(p),
    .r(r),
    .clk(clk),
    .reset(reset),
    .m(m)
);
light light1(
    .ta(ta),
    .tb(tb),
    .reset(reset),
    .clk(clk),
    .m(m),
    .la(la),
    .lb(lb)
);

endmodule
```

给出测试程序：

```
timeunit 1ps;
module testbench_430();

logic clk;
logic reset;
logic ta;
logic tb;
logic r;
logic p;
logic [1:0] la;
logic [1:0] lb;

controller dut (
    clk,reset,ta,tb,r,p,la,lb
);

initial begin
    clk = 0;
    forever #10 clk = ~clk;
end

initial begin
    reset = 1;
    ta = 1;
    tb = 1;
    r = 0;
    p = 0;

    #100;
    reset = 0;

    //test1:car from A
    #200;
    ta = 0;

    #200;
    ta = 1;

    //test2:change mode
    #300;
    p = 1;
    #40;
    p = 0;

    //test3:no car from B
    #300;
    tb = 0;
```

```
    //test4:reset
    #300;
    r = 1;
    #40;
    r = 0;

    //test5:combine
    #300;
    ta = 0;
    tb = 0;
    p = 1;
    #40;
    p = 0;

    #500;
    $stop;
end

endmodule
```

测试写的略有些麻烦，就是一点点穷举每个状态。下面给出仿真截图



T4.35

思路：这道题比上道题简单一些，只需要一个有限状态机。按照题目要求一点点写就行。

```
module woniu(
    input logic data,
    input logic clk,
    input logic reset,
    output logic smile
);

typedef enum logic [2:0] {s0,s1,s2,s3,s4} state;
state now , next;

always_ff @(posedge clk , posedge reset) begin
    if(reset == 0) now <= s0;
    else state <= next_state;
end

alwats_comb begin
    next = now;
    smile = 0;
    case(now)
        s0: begin
            if(data == 1) next = s1;
            else next = s0;
        end
        s1: begin
            if(data == 1) next = s2;
            else next = s0;
        end
        s2: begin
            if(data == 0) next = s3;
            else next = s4;
        end
        s3: begin
            if(data == 1) begin
                next = s1;
                smile = 1;
            end
            else next = s0;
        end
        s4: begin
            if(data == 0) begin
                next = s0;
                smile = 1;
            end
            else next = s4;
        end
        default: next = s0;
    endcase
```

```
    end

    endmodule
```

下面给出测试代码

```
timeunit 2ps;
module tb_woniu();

logic clk,reset,data,smile;

woniu dut(
    .clk(clk),
    .reset(reset),
    .data(data),
    .smile(smile)
);

initial begin
    clk = 0;
    forever #5 clk = ~clk;

    reset = 0;
    data = 0;
    #5;
    reset = 1;
    #5;

    //test1: 1101
    data = 1;#1;
    data = 1;#1;
    data = 0;#1;
    data = 1;#1;

    //test2: 1110
    data = 1;#1;
    data = 1;#1;
    data = 1;#1;
    data = 0;#1;

    //test3: 1100
    data = 1;#1;
    data = 1;#1;
    data = 0;#1;
    data = 0;#1;

    //test4: 111110
    data = 1;#1;
    data = 1;#1;
    data = 1;#1;
    data = 1;#1;
    data = 1;#1;
    data = 0;#1;
```

```
    //test5: 1101110
    data = 1; #1;
    data = 1; #1;
    data = 0; #1;
    data = 1; #1;
    data = 1; #1;
    data = 1; #1;
    data = 0; #1;
end

endmodule
```

T4.39

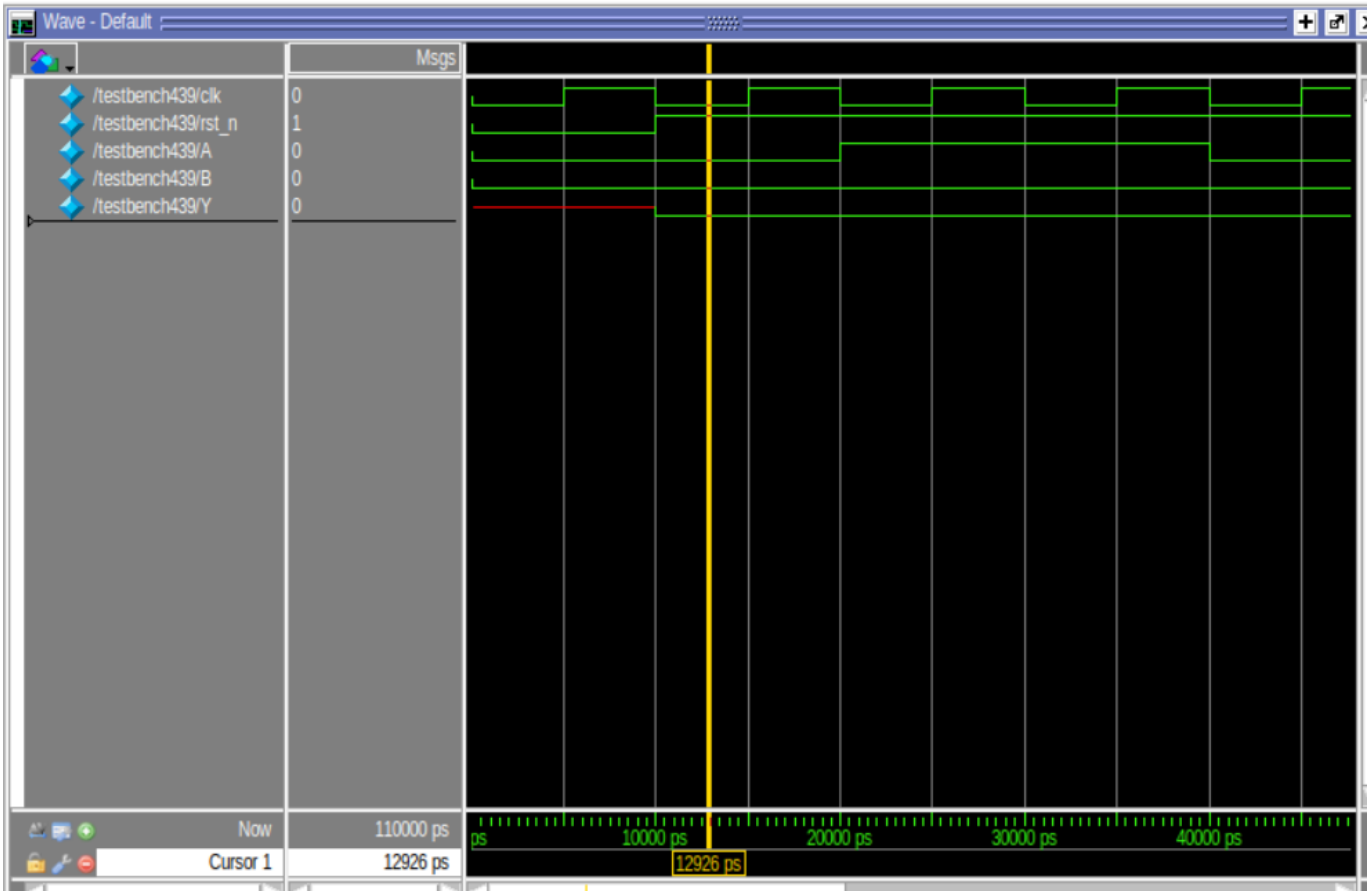
思路：按照题目描述写就行

```
module func439(
    input logic a,
    input logic b,
    input logic clk,
    input logic reset,
    output logic y
);

typedef enum logic [0:0] {s0,s1} state;
state now,next;

always_ff @(posedge clk,posedge reset) begin
    if(reset == 1) now <= s0;
    else now <= next;
end

always_comb begin
    case (now)
        s0: begin
            if(a == 1) begin
                next = s1;
                y=b;
            end
            else begin
                next = s0;
                y=0;
            end
        end
        s1: begin
            if(a == 1) begin
                next = s1;
                y=1;
            end
            else begin
                next = s0;
                y=b;
            end
        end
    endcase
end

endmodule
```

下面给出测试代码。由于与前几次测试大同小异，故此测试代码是借助AI生成的。

```
`timescale 1ns/1ps

module testbench439();
    logic clk;      // 时钟信号
    logic rst_n;    // 异步复位信号，低电平有效
    logic A;  // 输入数据位
    logic B;
    logic Y;   // 检测输出信号

    func439 dut (
        .clk(clk),
        .reset(rst_n),
        .a(A),
        .b(B),
        .y(Y)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // 测试序列
    initial begin
        rst_n = 0;
        A = 0;
        B = 0;
        #10;
        rst_n = 1;
        #10;

        A=1; #20;
        A=0; #20;
        B=1; #10;
        A=1; #10;
        A=0; #10;

        #20;
        $finish;
    end

endmodule
```

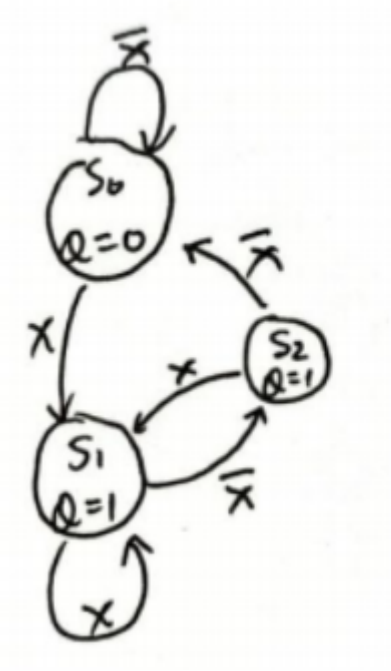
下面给出仿真截图



这里可以看出，最开始的一个时钟周期时候Y是高阻X状态。这可能是因为没有初始化造成的latch。

T4.42

思路：先翻出上周的作业，找到状态转换图



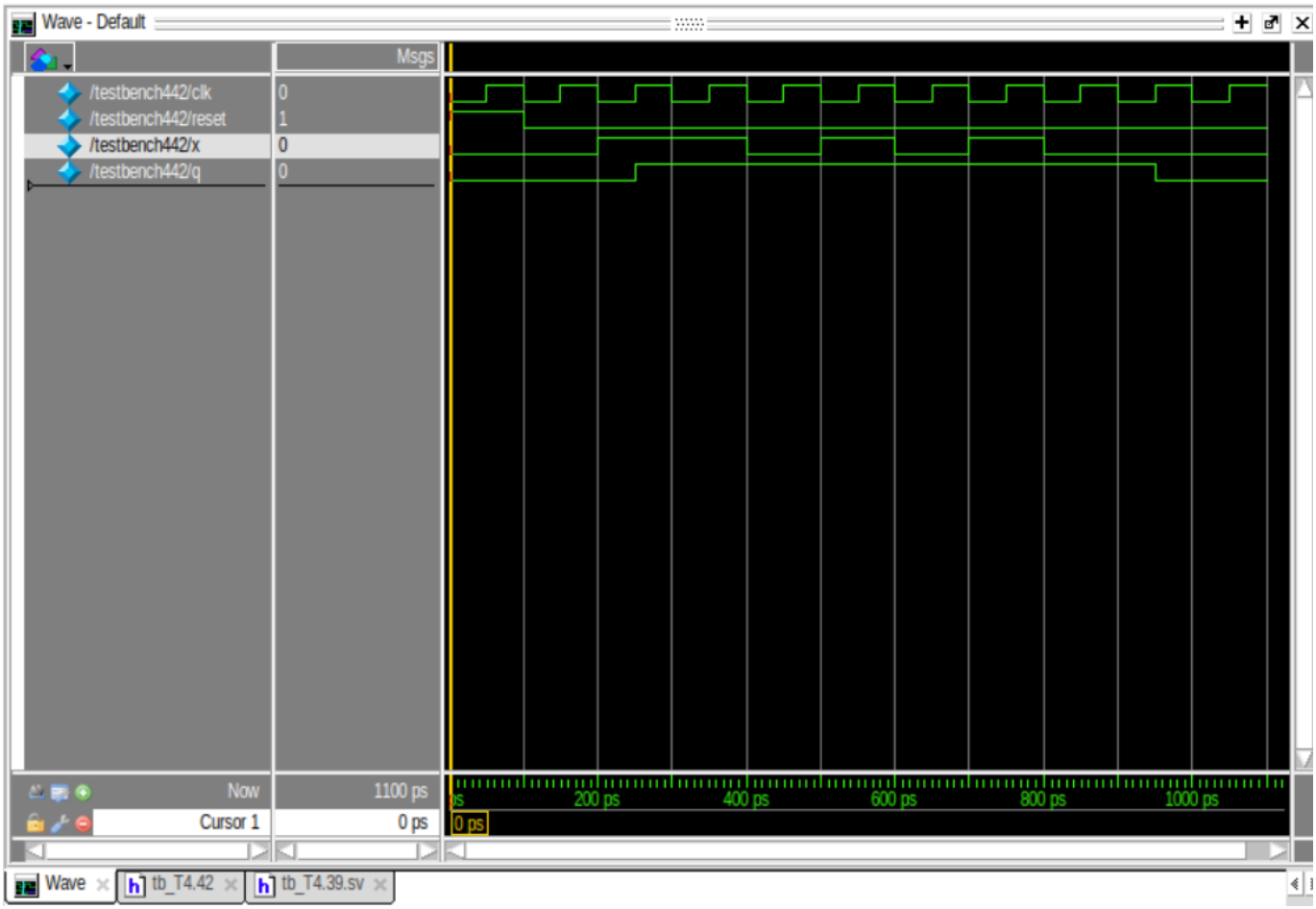
然后对这这个图写就行，本质上和前几个题没区别。

```
module func442(
    input logic x,
    input logic clk,
    input logic reset,
    output logic q
);

typedef enum logic [1:0] {s0,s1,s2} state;
state now,next;

always_ff @(posedge clk,posedge reset) begin
    if(reset == 1) now <= s0;
    else now <= next;
end

always_comb begin
    case (now)
        s0: begin
            if(x == 0) next = s0;
            else next = s1;
        end
        s1: begin
            if(x == 0) next = s2;
            else next = s1;
        end
        s2: begin
            if(x == 0) next = s0;
            else next = s1;
        end
    endcase
end

always_comb begin
    case (now)
        s0: q=0;
        s1: q=1;
        s2: q=1;
    endcase
end

endmodule
```

下面给出测试代码

```
`timescale 10ps/1ps

module testbench442();

    logic clk;
    logic reset;
    logic x;
    logic q;

    // 实例化 DUT
    func442 dut (
        .x(x),
        .clk(clk),
        .reset(reset),
        .q(q)
    );

    // 生成时钟
    initial clk = 0;
    always #5 clk = ~clk; // 10ns周期

    // 测试序列
    initial begin
        $display("Time\tReset\tX\tQ\tState");
        $monitor("%0t\t%b\t%b\t%b", $time, reset, x, q);

        // 初始化
        reset = 1; x = 0;
        #10;
        reset = 0;

        // 测试输入序列
        #10 x = 1;   // s0 -> s1
        #10 x = 1;   // s1 -> s2
        #10 x = 0;   // s2 -> s0
        #10 x = 1;   // s0 -> s1
        #10 x = 0;   // s1 -> s1
        #10 x = 1;   // s1 -> s2
        #10 x = 0;   // s2 -> s0
        #10 x = 0;   // s0 -> s0
        #20 $finish;
    end

endmodule
```

下面给出仿真截图



T4.48&4.49

题目4.48中，由于最初使用的是非阻塞的赋值<=，所以不会立即生效，即y读取的还是上一个周期的x值（可以理解成对y的赋值和对x的赋值“同时”进行，不分先后）。但当把非阻塞的赋值<=改成阻塞的赋值=之后，两段代码中赋值语句先后顺序不同，会导致运行结果有差别。

T4.50

(a)欲设计一个锁存器latch，应当写成always_latch

(b) 敏感变量列表少了b。应当简单写成always_comb

(c)多路选择器应当选用组合逻辑而不是时序逻辑

(d)应当使用非阻塞的赋值<=

(e)直接给出改正后的版本

```
module FSM(input  logic clk,
           input  logic a,
           output logic out1, out2);
    logic state;


    always_ff @(posedge clk) begin
        if (state == 0) begin
            if (a) state <= 1;
        end else begin
            if (~a) state <= 0;
        end
    end


    always_comb begin
        out1 = 0;
        out2 = 0;
        if (state == 0) out1 = 1;
        else out2 = 1;
    end
endmodule
```

(f)当a[i]的所有位都为0时，y没有值。换言之，应当加入default: y=0

(g)没写default

(h)两个三态缓冲器冲突了。控制端应当分别用s和~s

(i)缺少reset和clk的敏感

(j)组合逻辑应使用阻塞的赋值=